ACADEMIC CENTRAL

PRACTICE EXAMINATION

NC(V) LEVEL 4

# COMPUTER PROGRAMMING

## PAPER 1 · PRACTICE A · MARKING GUIDELINE

**MARKS: 100**

**Marking guideline for the Academic Central NC(V) Level 4 Computer Programming paper 1 · practice a.**

Written by Academic Central for practice, following the NC(V) subject guidelines for NC(V) Level 4. It is not a Department of Higher Education and Training marking guideline.

**How to mark:** each ✓ is one mark. Code may be in any language or pseudocode. Mark the logic, not the syntax. The ticks show where marks are given; accept other correct solutions.

## SECTION A

## QUESTION 1

**1.1.1** D ✓
**1.1.2** B ✓
**1.1.3** C ✓
**1.1.4** A ✓
**1.1.5** B ✓
**1.1.6** D ✓
**1.1.7** A ✓
**1.1.8** C ✓
**1.1.9** C ✓
**1.1.10** A ✓

(10)

**1.2.1** True ✓
**1.2.2** True ✓
**1.2.3** True ✓
**1.2.4** False ✓
**1.2.5** True ✓
**1.2.6** False ✓
**1.2.7** True ✓
**1.2.8** False ✓
**1.2.9** True ✓
**1.2.10** True ✓

(10)

**1.3.1** H ✓
**1.3.2** J ✓
**1.3.3** G ✓
**1.3.4** E ✓
**1.3.5** B ✓
**1.3.6** I ✓
**1.3.7** F ✓
**1.3.8** D ✓
**1.3.9** C ✓
**1.3.10** A ✓

(10)

**TOTAL SECTION A: [30]**

## SECTION B

## QUESTION 2

2.1.1 – means private (only accessible inside the class) ✓; + means public (accessible from outside) ✓ (2)

2.1.2 Example:

```
class Student:                                          ✓
    def __init__(self, studentNo, name, feesOwed):      ✓
        self.__studentNo = studentNo                    ✓ (private attributes)
        self.__name = name
        self.__feesOwed = feesOwed                      ✓
    def getName(self):
        return self.__name                              ✓
```

(5)

2.1.3 Example:

```
    def makePayment(self, amount):                          ✓
        if amount > 0 and amount <= self.__feesOwed:        ✓✓
            self.__feesOwed = self.__feesOwed - amount      ✓
        else:
            print("Invalid payment")                        ✓
```

(5)

2.1.4 def isPaidUp(self): return self.__feesOwed == 0 ✓✓ (accept <= 0) (2)

2.1.5 def toString(self): ✓ return self.__studentNo + " " + self.__name ✓ + " owes R" + format(self.__feesOwed, ".2f") ✓ (3)

2.1.6 Example:

```
s = Student("202401234", "Thabo Nkosi", 4500)   ✓✓
s.makePayment(3000)                             ✓
print(s.toString())                             ✓
# Output: 202401234 Thabo Nkosi owes R1500.00
```

(4)

**2.2** Any TWO: code in Student is reused, not rewritten ✓; changes to Student automatically apply to Bursar ✓; it models the real relationship (a bursar is a student); easier to maintain (2)

**2.3** So that it cannot be changed directly from outside the class ✓; changes must go through makePayment, which checks that the payment is valid (data integrity) ✓. (2)

**[25]**

## QUESTION 3

**3.1** Example:

```
def countVowels(word):                  ✓
    count = 0                           ✓
    for ch in word.lower():             ✓
        if ch in "aeiou":               ✓
            count = count + 1
    return count                        ✓
```

(5)

3.2.1 Pass 1: [12, 27, 8, 19, 45] ✓; pass 2: [12, 8, 19, 27, 45] ✓; pass 3: [8, 12, 19, 27, 45] ✓; pass 4: [8, 12, 19, 27, 45] (no swaps – sorted) ✓ (4)

3.2.2 Example:

```
n = len(nums)
for i in range(n - 1):                          ✓
    for j in range(n - 1 - i):                  ✓
        if nums[j] > nums[j + 1]:               ✓
            temp = nums[j]                      ✓
            nums[j] = nums[j + 1]               ✓
            nums[j + 1] = temp                  ✓
```

(6)

**3.3** Sorted list [8, 12, 19, 27, 45]: middle element (index 2) is 19 ✓; 27 > 19, so search the right half [27, 45] ✓; new middle (index 3) is 27 ✓; found at index 3 (2 comparisons) ✓. (4)

**3.4** Advantage: much faster for large lists (halves the list each time) ✓; disadvantage: the list must be sorted first / more complex to code ✓ (2)

**[21]**

## QUESTION 4

4.1.1 Example:

```
total = 0
count = 0                                       ✓
f = open("results.txt", "r")                    ✓
for line in f:                                  ✓
    parts = line.strip().split(",")             ✓
    name = parts[1]
    mark = int(parts[2])                        ✓
    if mark >= 50:
        print(name)                             ✓
    total = total + mark
    count = count + 1                           ✓
f.close()
print("Average:", round(total / count, 1))      ✓
```

(8)

| | | |
|---|---|---|
| 4.1.2 | try: ✓ (code from 4.1.1) except FileNotFoundError: print("The results file could not be found.") ✓ except ValueError: print("A mark in the file is not a valid number.") ✓; sensible placement / friendly messages ✓ | (4) |
| **4.2** | f = open("results.txt", "a") ✓; f.write("202401237,Lindiwe Zulu,74\n") ✓; f.close() ✓ | (3) |
| **4.3** | A text file stores data as plain lines of text ✓ – simple, small, readable in any editor ✓; a database stores data in structured tables with relationships ✓ – fast searching / queries (SQL), many users at once, data validation and security ✓. | (4) |
| **4.4** | Users make mistakes / unexpected situations happen ✓; exception handling prevents crashes and shows helpful messages, so the program keeps running and data is not lost ✓. | (2) |
| **4.5** | Any TWO ✓✓ (+ ✓ for a clear explanation): a user types text where a number is expected; division by zero; a network or database connection fails; an index outside the list range; low memory / disk full | (3) |

**[24]**

**TOTAL SECTION B: [70]**

**TOTAL: 100**

**END OF MARKING GUIDELINE**